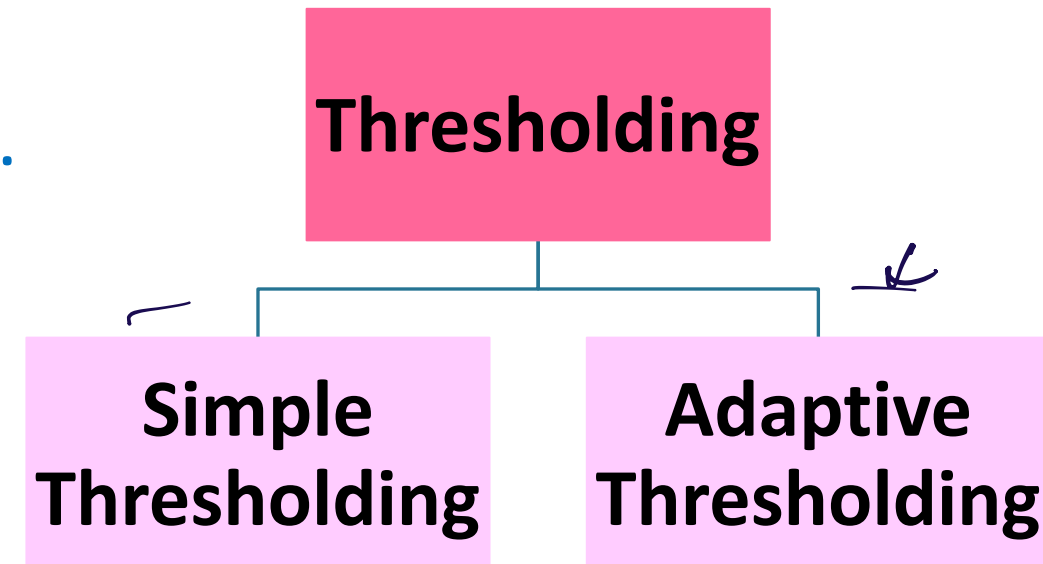


Thresholding

- Thresholding is an image enhancement technique that converts a **grayscale image into a binary image** (black and white) by selecting a **cut-off pixel value (threshold)**.
- It's like saying: "If a pixel is bright enough, keep it white. Otherwise, make it black."
- Simple Thresholding:** Imagine 3×3 grayscale image:
- If we set a **threshold of 100**, then: $\text{Pixels} \geq 100 \rightarrow 255$ (white) and $\text{Pixels} < 100 \rightarrow 0$ (black)

			Transformed Pixels		
80	100	90	0	255	0
120	150	110	255	255	25
85	130	140	0	255	25



Thresholding

- **Adaptive thresholding:** Unlike **simple thresholding**, which uses **one fixed threshold** for the whole image, **adaptive thresholding** calculates a **different threshold for each pixel** — based on its **neighborhood**.
- It adapts to **changing lighting** across the image.
- **Formula:** $T(x, y) = \text{Mean of neighborhood around } (x, y) - C$, Where:
- $T(x, y)$: Threshold at pixel (x, y)
 - Neighborhood = block size (e.g., 3×3)
 - C : A constant subtracted to fine-tune the result
- We'll go row-by-row. For each pixel, we take the 3×3 region around it, calculate the **mean**, subtract **$C = 5$** , then apply the rule:

$$\text{Output} = \begin{cases} 255 & \text{if } \text{centre pixel} \geq \text{mean} - 5 \\ 0 & \text{Otherwise} \end{cases}$$

Thresholding ✱

60	90	100
70	100	130
85	115	45

Calculating for Pixel (0, 0): value = 60

0	0	0
0	60	90
0	70	100



[5]

$$\text{Sum} = 320, \text{Mean} = 320 / 9 \approx 35.56$$

$$\text{Threshold} = 35.56 - 5 = 30.56$$

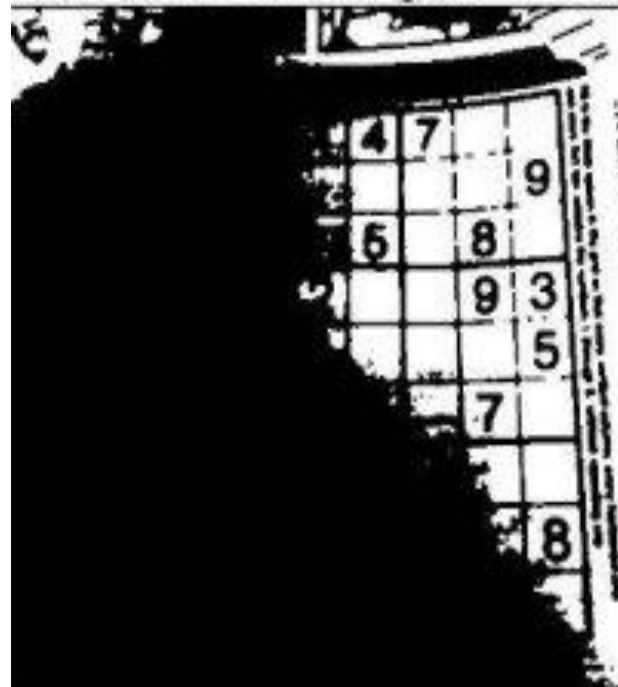
$$\text{Since } 60 \geq 30.56 \rightarrow \text{Output} = 255$$

Now, 60 will replace to 255. Similarly, the procedure is repeated for every pixel of the image.

Thresholding



Original
Image



Simple
Thresholding

[6]



Adaptive
Thresholding

Otsu Thresholding

Idea:

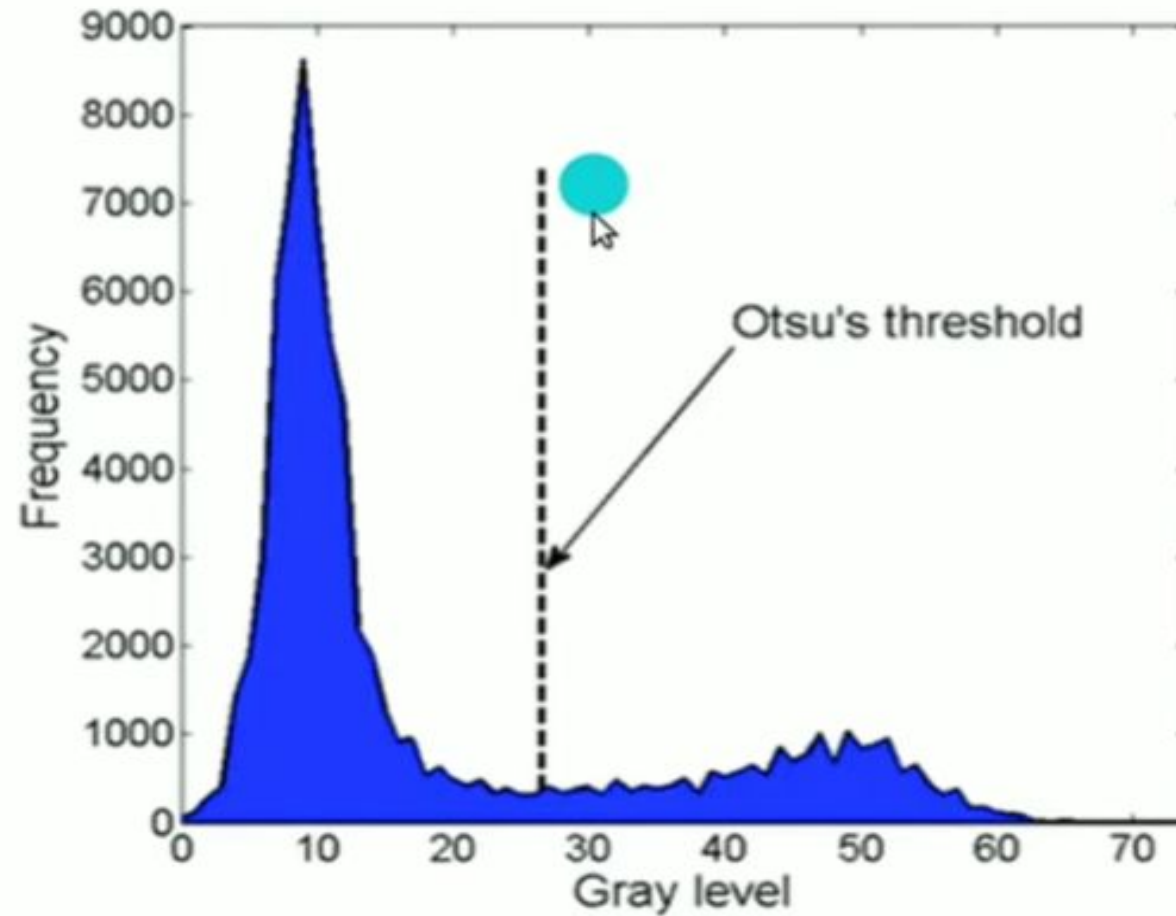
Automatically determine an **optimal global threshold** by **minimizing intra-class variance** (or **maximizing inter-class variance**).

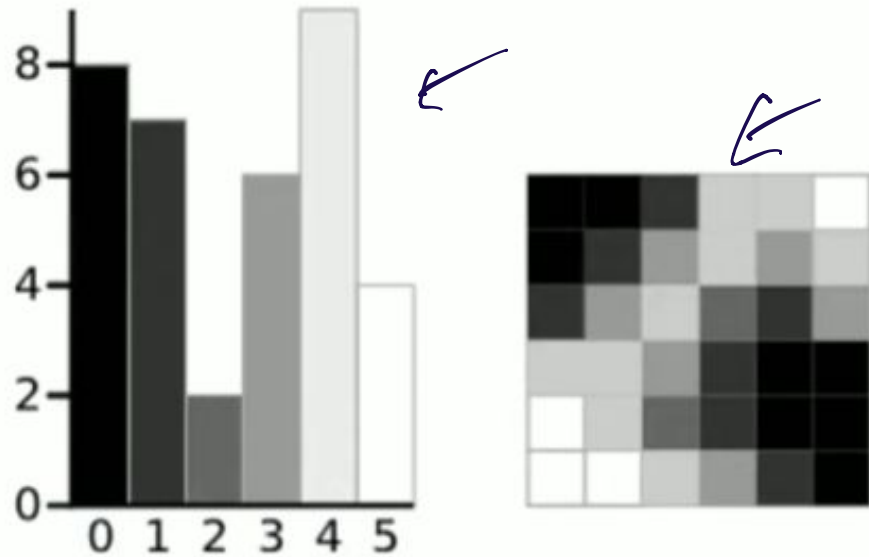
How it works:

- Assumes the image has a **bimodal histogram** (foreground + background).
- Calculates the threshold value that **best separates these two classes statistically**.

Best for: Images with **distinct bimodal intensity distributions**. F/B

Otsu Thresholding



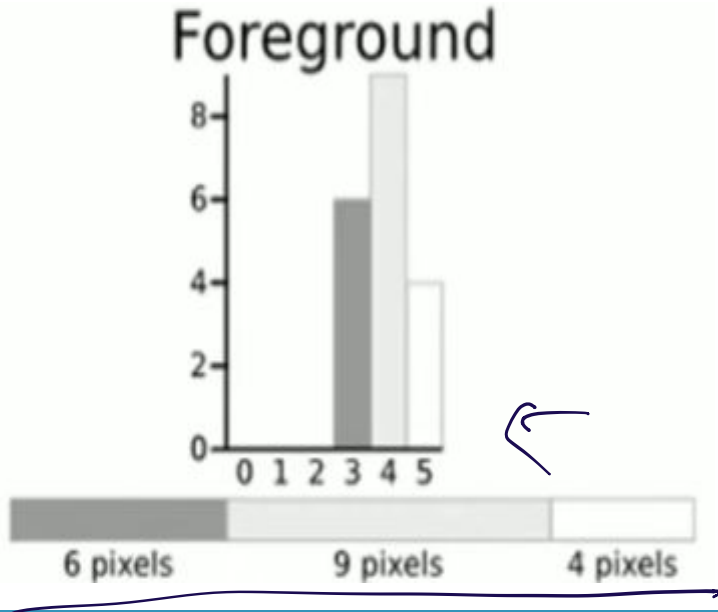


$$\text{Weight } W_f = \frac{6 + 9 + 4}{36} = 0.5278$$

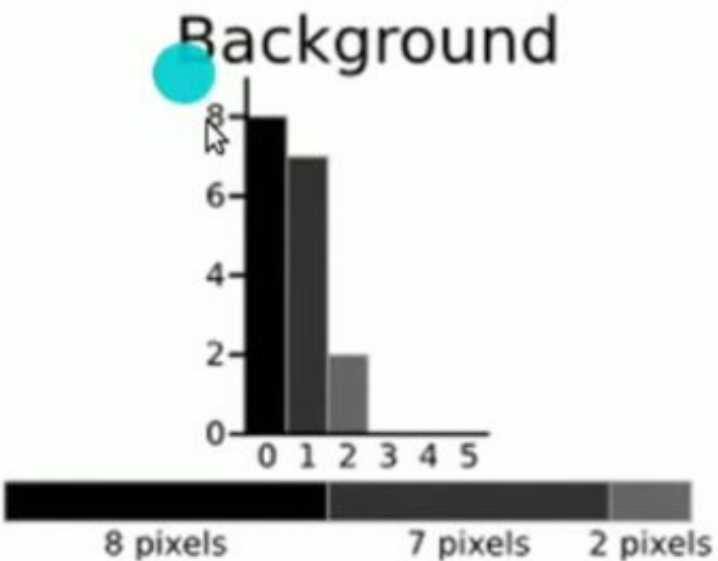
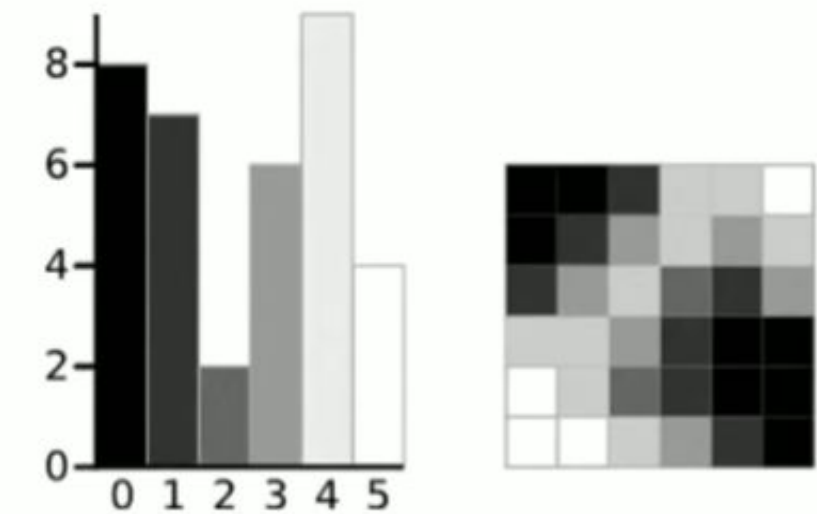
Handwritten arrow points to the denominator 36.

$$\text{Mean } \mu_f = \frac{(3 \times 6) + (4 \times 9) + (5 \times 4)}{19} = 3.8947$$

$$\begin{aligned} \text{Variance } \sigma_f^2 &= \frac{((3 - 3.8947)^2 \times 6) + ((4 - 3.8947)^2 \times 9) + ((5 - 3.8947)^2 \times 4)}{19} \\ &= \frac{(4.8033 \times 6) + (0.0997 \times 9) + (4.8864 \times 4)}{19} \\ &= 0.5152 \end{aligned}$$



Otsu's Thresholding Mathematics



Weight $W_b = \frac{8 + 7 + 2}{36} = 0.4722$

Mean $\mu_b = \frac{(0 \times 8) + (1 \times 7) + (2 \times 2)}{17} = 0.6471$

Variance $\sigma_b^2 = \frac{((0 - 0.6471)^2 \times 8) + ((1 - 0.6471)^2 \times 7) + ((2 - 0.6471)^2 \times 2)}{17}$

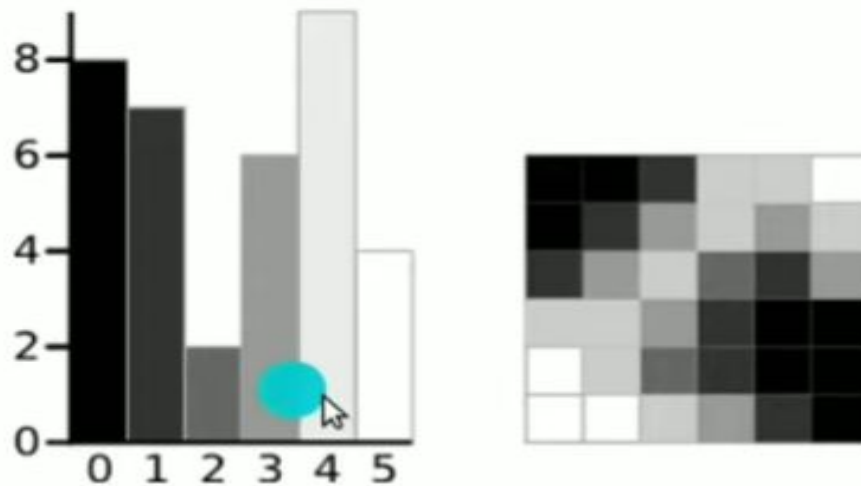
$= \frac{(0.4187 \times 8) + (0.1246 \times 7) + (1.8304 \times 2)}{17}$

$= 0.4637$

Otsu Thresholding

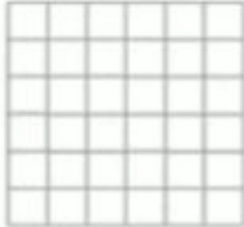
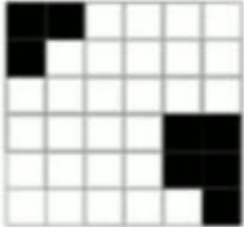
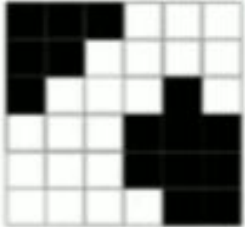
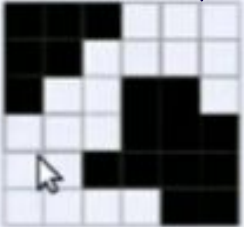
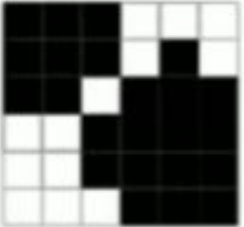
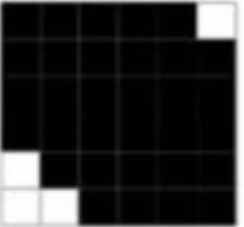
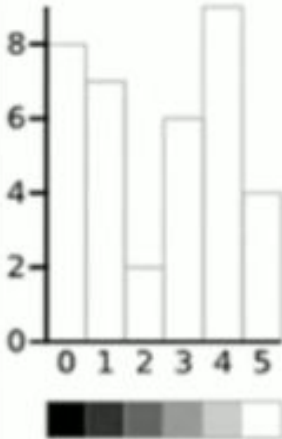
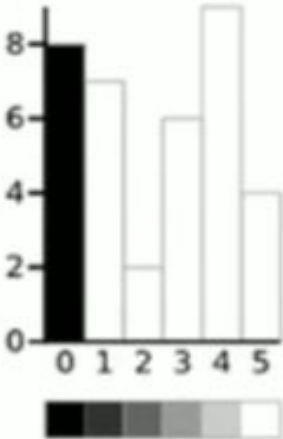
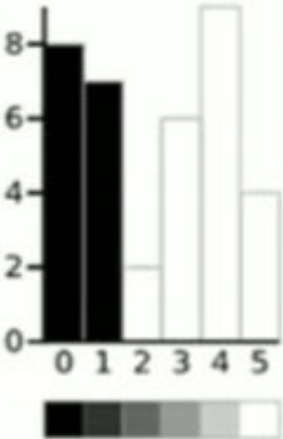
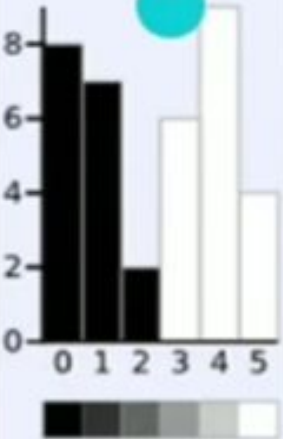
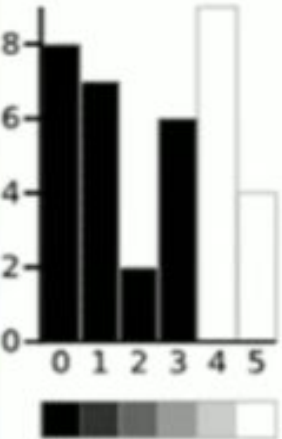
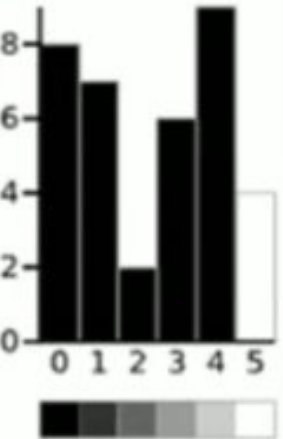


Otsu's Thresholding Mathematics



Within Class Variance

$$\sigma_W^2 = W_b \sigma_b^2 + W_f \sigma_f^2 = 0.4722 * 0.4637 + 0.5278 * 0.5152 = 0.4909$$

Threshold	<u>T=0</u>	T=1	T=2	<u>T=3</u>	T=4	<u>T=5</u>
						
						
Weight, Background	$W_b = 0$	$W_b = 0.222$	$W_b = 0.4167$	$W_b = 0.4722$	$W_b = 0.6389$	$W_b = 0.8889$
Mean, Background	$M_b = 0$	$M_b = 0$	$M_b = 0.4667$	$M_b = 0.6471$	$M_b = 1.2609$	$M_b = 2.0313$
Variance, Background	$\sigma^2_b = 0$	$\sigma^2_b = 0$	$\sigma^2_b = 0.2489$	$\sigma^2_b = 0.4637$	$\sigma^2_b = 1.4102$	$\sigma^2_b = 2.5303$
Weight, Foreground	$W_f = 1$	$W_f = 0.7778$	$W_f = 0.5833$	$W_f = 0.5278$	$W_f = 0.3611$	$W_f = 0.1111$
Mean, Foreground	$M_f = 2.3611$	$M_f = 3.0357$	$M_f = 3.7143$	$M_f = 3.8947$	$M_f = 4.3077$	$M_f = 5.000$
Variance, Foreground	$\sigma^2_f = 3.1196$	$\sigma^2_f = 1.9639$	$\sigma^2_f = 0.7755$	$\sigma^2_f = 0.5152$	$\sigma^2_f = 0.2130$	$\sigma^2_f = 0$
Within Class Variance	$\sigma^2_W = 3.1196$	$\sigma^2_W = 1.5268$	$\sigma^2_W = 0.5561$	$\sigma^2_W = 0.4909$	$\sigma^2_W = 0.9779$	$\sigma^2_W = 2.2491$

(thresholding)
 Otsu's segmentation algorithm automatically clusters pixels into two groups: background and foreground. The main idea of Otsu's algorithm is to find threshold that would maximize between-class variance and minimize withinclass variance.

First step is to create a histogram of pixel values. Since we are taking 8-bit image, there are 256 possible values for pixels. We compute histogram h with 256 bins where the height of each bin corresponds to number of pixels that have that pixel value (from 0 to 255).

Algorithm

The algorithm iteratively searches for the threshold that minimizes the within-class variance, defined as a weighted sum of variances of the two classes (background and foreground). The colors in grayscale are usually between 0-255 (0-1 in case of float). So, If we choose a threshold of 100, then all the pixels with values less than 100 becomes the background and all pixels with values greater than or equal to 100 becomes the foreground of the image.

The formula for finding the within-class variance at any threshold t is given by:

$$\sigma^2(t) = \omega_{bg}(t)\sigma_{bg}^2(t) + \omega_{fg}(t)\sigma_{fg}^2(t) \quad (1)$$

where $\omega_{bg}(t)$ and $\omega_{fg}(t)$ represents the probability of number of pixels for each class at threshold t and σ^2 represents the variance of color values.

To understand what this probability means, Let,

P_{all} be the total count of pixels in an image,

$P_{BG}(t)$ be the count of background pixels at threshold t ,

$P_{FG}(t)$ be the count of foreground pixels at threshold t

So the weights are given by,

$$\omega_{bg}(t) = \frac{P_{BG}(t)}{P_{all}}$$

$$\omega_{fg}(t) = \frac{P_{FG}(t)}{P_{all}}$$

$$\omega_{fg}(t) =$$

The variance can be calculated using the below formula:

$$\sigma^2(t) = \frac{\sum (x_i - \bar{x})^2}{N-1} \leftarrow$$

where,

x_i is the value of pixel at i in the group (bg or fg)

$\bar{x}$ is the means of pixel values in the group (bg or fg)

N is the number of pixels.

Now, Let's understand the formula by finding the within-class variance at one threshold, $T=100$

120	120	21	22
25	26	27	160
180	190	123	145
165	175	23	24

Image with pixel color values

For the above image, at $T=100$, we will get the background and foreground as shown below:

120	120		
			160
180	190	123	145
165	175		

Here,

$P_{all}=16$

→ $P_{BG}=7$

→ $P_{FG}=9$

Our weights will be,

$$\underline{\omega_{bg}(t)} = \frac{P_{BG}(t)}{P_{all}} = 7/16 = \underline{0.44}$$

$$\underline{\omega_{fg}(t)} = \frac{P_{FG}(t)}{P_{all}} = 9/16 = \underline{0.56}$$

Now to find the variance, we find the mean first.

$$\begin{aligned} \rightarrow \bar{x}_{bg} &= \frac{21+22+25+26+27+23+24}{7} = \underline{24} \\ \rightarrow \bar{x}_{fg} &= \frac{120+120+160+180+190+123+145+165+175}{9} = 153.1 \end{aligned}$$

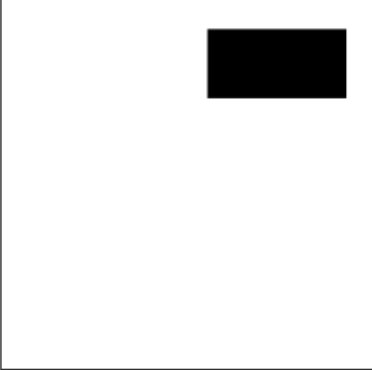
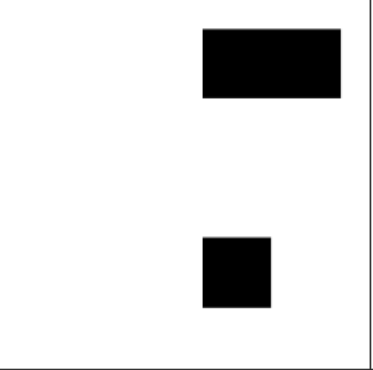
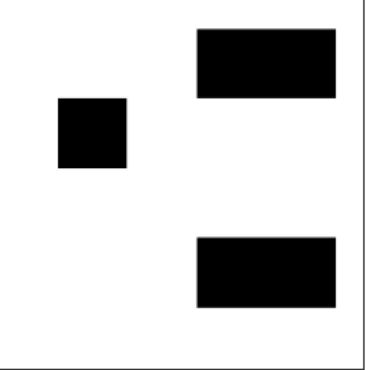
The variances are given by,

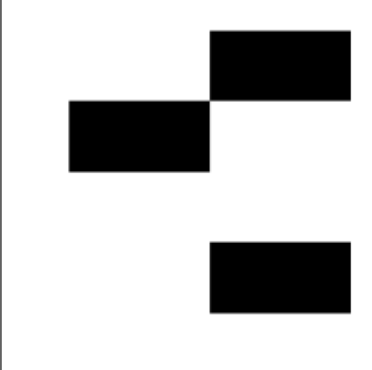
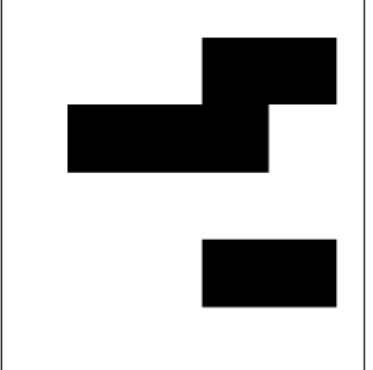
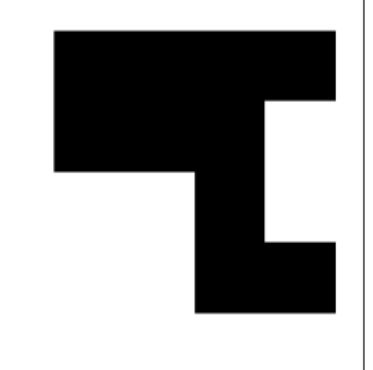
$$\begin{aligned} \rightarrow \sigma_{bg}^2(t=100) &= \frac{(21-24)^2 + (22-24)^2 + \dots + (24-24)^2}{7} = 4.0 \\ \rightarrow \sigma_{fg}^2(t=100) &= \frac{(120-153.1)^2 + (120-153.1)^2 + \dots + (175-153.1)^2}{9} = \underline{657.43} \end{aligned}$$

Substituting everything in equation (1) we get,

$$\underline{\sigma^2(t=100)} = 0.44 * 4.0 + 0.56 * 657.43 = \underline{369.9208} \leftarrow$$

variance

		
T=22, $\sigma^2 = 4092.58$	T=23, $\sigma^2 = 3667.60$	T=25, $\sigma^2 = 2642.35$

		
T=26, $\sigma^2 = 2009.93$	T=28, $\sigma^2 = 371.55$	T=124, $\sigma^2 = 1316.48$

The value of variance remains the same from 28 and 120.

If you see the above variances, its least at T=28 or more precicely between 28 to 120.

Thus our Otsu threshold = 28.